BayRC Reference Manual

July 14, 2026

BayRC-package *BayRC: Bayesian Framework for Comparative Circadian Genomics*

Description

BayRC (Bayesian Rhythmicity Comparison) is a unified statistical framework for comparing and interpreting circadian rhythms across biological conditions: age, disease state, tissue, species, or sex. It jointly infers gene-level rhythmicity and phase, computes posterior probabilities of rhythmic and phase concordance, and classifies rhythmic gain, loss, conservation, and direction-specific phase shifts under Bayesian false discovery rate (BFDR) control. It further supports pathway-level enrichment and genome-wide concordance scoring, providing a unified, uncertainty-aware framework for comparative circadian analysis across tissues, species, and disease contexts.

Main functions

MCMC core (paper Sec. 2.1)
- CB_init_single
- CBt_init_single
- CB_MCMC_single_rj_slice
- CB_getAllEst
- CBt_sim_data
- circular_HDI
- circular_median

Gene-level biomarker detection with BFDR control (paper Sec. 2.2)
- match_symbols
- bfdr_from_posterior
- detect_rhy
- summarize_bay
- transition_classify
- phase_infer

Pathway-level rhythmic enrichment (paper Sec. 2.3)
- pathSelect
- plot_heatmap
- multi_conservation_pathway
- multi_conservation_pathway_bootstrap

Genome-wide concordance summary (paper Sec. 2.4)
- multi_conservation
- pairwise_concordance

Cross-species alignment
- match_homologs
- merge_mcmc

Author(s)

Maintainer: Thien Quy Pham <quythien14@gmail.com>

Authors:

- Xiangning Xue
- Xingjian Zhang
- Guan Yu
- Liangliang Zhang
- George C Tseng <ctseng@pitt.edu>

CB_init_single — *Initialize single-species BayRC MCMC chain*

Description

Produces starting values for the BayRC MCMC sampler by fitting an ordinary least-squares cosinor model to every gene. OLS-derived MESOR, amplitude, phase, and residual variance are used as initial values; rhythmicity is initialised from an F-test at alpha = 0.05. When `FitCosinor = FALSE`, parameters are drawn from their priors.

Usage

```
CB_init_single(
  Data.list = list(data = as.data.frame(a.sim.dat$data[[1]]$dat), time =
    a.sim.dat$data[[1]]$x$time, gname = paste0("G",
    seq_len(nrow(a.sim.dat$data[[1]]$dat)))),
  P = 24,
  FitCosinor = TRUE,
  mu_M = 0,
  sigma_M = 10,
  mu_A = 1,
  sigma_A = 10,
  seed = 15213
)
```

Arguments

`Data.list`	A named list with three elements: `data` (G x N data.frame of expression values), `time` (length-N numeric Zeitgeber time vector in hours), and `gname` (length-G character vector of gene identifiers).
`P`	Numeric; circadian period in hours (default 24).
`FitCosinor`	Logical; if `TRUE` (default) use OLS cosinor estimates; if `FALSE` draw from priors.
`mu_M`	Numeric; prior mean for MESOR (default 0).
`sigma_M`	Numeric; prior standard deviation for MESOR (default 10).
`mu_A`	Numeric; prior mean for amplitude (default 1).
`sigma_A`	Numeric; prior standard deviation for amplitude (default 10).
`seed`	Integer; random seed (default 15213).

Details

Initialize MCMC chain for a single circadian dataset

Value

A named list with elements `rho` (length-G logical), `M` (length-G MESOR), `A` (length-G amplitude), `phi` (length-G acrophase in hours), and `sigma` (length-G residual variance). Suitable for passing directly to `CB_MCMC_single_rj_slice` as `Init.value`.

Examples

```
## Not run:
sim  <- CBt_sim_data()
dat  <- list(data  = as.data.frame(sim$data[[1]]$dat),
             time  = sim$data[[1]]$x$time,
             gname = paste0("G", seq_len(nrow(sim$data[[1]]$dat))))
init <- CB_init_single(dat)

## End(Not run)
```

`CBt_init_single` *Initialize time-error BayRC MCMC chain*

Description

Produces starting values for the CBt MCMC sampler, which models Zeitgeber-time measurement error (t_p). Parameter initialisation follows the same OLS cosinor approach as `CB_init_single` but also initialises the time-error latent variable `t_p` to zero.

Usage

```
CBt_init_single(
  Data.list = list(data = as.data.frame(a.sim.dat$data[[1]]$dat), time =
    a.sim.dat$data[[1]]$x$time, gname = paste0("G",
    seq_len(nrow(a.sim.dat$data[[1]]$dat)))),
  P = 24,
  FitCosinor = TRUE,
  mu_M = 0,
  sigma_M = 10,
  mu_A = 1,
  sigma_A = 10,
  seed = 15213
)
```

Arguments

`Data.list`	A named list with three elements: `data` (G x N data.frame of expression values), `time` (length-N numeric Zeitgeber time vector in hours), and `gname` (length-G character gene identifiers).
`P`	Numeric; circadian period in hours (default 24).

FitCosinor	Logical; if TRUE (default) use OLS cosinor estimates; if FALSE draw from priors.
mu_M	Numeric; prior mean for MESOR (default 0).
sigma_M	Numeric; prior standard deviation for MESOR (default 10).
mu_A	Numeric; prior mean for amplitude (default 1).
sigma_A	Numeric; prior standard deviation for amplitude (default 10).
seed	Integer; random seed (default 15213).

Details

Initialize MCMC chain for a dataset with Zeitgeber-time uncertainty

Value

A named list with elements rho, M, A, phi, sigma (same as CB_init_single), plus t_p (length-N vector of initial time-error values, set to 0). Suitable for passing to CBt_MCMC_single as Init.value.

Examples

```
## Not run:
sim  <- CBt_sim_data()
dat  <- list(data  = as.data.frame(sim$data[[1]]$dat),
             time  = sim$data[[1]]$x$time,
             gname = paste0("G", seq_len(nrow(sim$data[[1]]$dat))))
init <- CBt_init_single(dat)

## End(Not run)
```

CB_MCMC_single_rj_slice

CB MCMC single reversible-jump slice sampler

Description

Runs the BayRC RJMCMC sampler for a single-species/condition circadian expression dataset. Within each iteration the sampler proposes model-space jumps (rho: 0/1 rhythmicity indicator) via an RJMCMC acceptance ratio and updates amplitude, phase, MESOR, and residual variance via Gibbs or slice steps.

Usage

```
CB_MCMC_single_rj_slice(
  Data.list,
  Init.value,
  P = 24,
  iteration = 3000,
  thin = 20,
  n.burn = 1000,
  seed = 15213,
  diagnostics = FALSE,
```

```
    p_rhythmic = rep(0.2, 100),
    rj.p.stay = 0.5,
    A_prior = "Jeffreys_OLS_condi",
    mu_A = 1,
    sigma_A = 10^2,
    A.min = 0,
    A_wb_beta2 = 2,
    A_gm_shape = 1.99,
    A_gm_rate = 0.5,
    rj.phi = TRUE,
    rj.A = TRUE,
    mu_M = 0,
    sigma_M = 10^2,
    sigma_prior_v = 4,
    sigma_prior_s = 1,
    save.file = "MCMC_save.rds",
    save.file2 = "MCMC_save.rds"
)
```

Arguments

Data.list	A named list with three elements: data (G x N data.frame of expression values), time (length-N numeric Zeitgeber time vector in hours), and gname (length-G character vector of gene identifiers).
Init.value	List returned by CB_init_single containing starting values for rho, M, A, phi, and sigma.
P	Numeric; circadian period in hours (default 24).
iteration	Integer; total number of MCMC iterations including burn-in (default 3000).
thin	Integer; thinning interval; every thin-th post-burn-in sample is stored (default 20).
n.burn	Integer; number of burn-in iterations discarded before storing samples (default 1000).
seed	Integer; random seed for reproducibility (default 15213).
p_rhythmic	Numeric vector of length G; prior probability Pr(rho_g = 1) for each gene. Enters the RJMCMC log-prior odds as log(p / (1 - p)). Default is rep(0.2, 100).
rj.p.stay	Numeric in (0, 1); probability of skipping the between-model move (staying in the current model) at each iteration (default 0.5).
A_prior	Character; prior on amplitude. One of "Jeffreys_OLS_condi", "trunc_Normal", "Jeffreys", "Jeffreys_ridge", "sq_expo", or "gamma" (default "Jeffreys_OLS_condi").
mu_A	Numeric; mean of the truncated-Normal amplitude prior when A_prior = "trunc_Normal" (default 1).
sigma_A	Numeric; variance of the truncated-Normal amplitude prior (default 10^2).
A.min	Numeric; lower truncation bound for amplitude (default 0).
A_wb_beta2	Numeric; beta parameter for the squared-exponential amplitude prior sq_expo (default 2).
A_gm_shape	Numeric; shape parameter for the gamma amplitude prior; values <= 2 give a flatter prior (default 1.99).

`A_gm_rate` Numeric; rate parameter for the gamma amplitude prior; smaller values give a flatter prior (default 0.5).

`rj.phi` Logical; if `TRUE`, phase phi is jointly proposed in the RJMCMC birth/death step (default `TRUE`).

`rj.A` Logical; if `TRUE`, amplitude A is jointly proposed in the RJMCMC birth/death step (default `TRUE`).

`mu_M` Numeric; prior mean for MESOR (default 0).

`sigma_M` Numeric; prior variance for MESOR (default `10^2`).

`sigma_prior_v` Numeric; degrees-of-freedom hyperparameter of the inverse-gamma prior on residual variance (default 4).

`sigma_prior_s` Numeric; scale hyperparameter of the inverse-gamma prior on residual variance (default 1).

`save.file` Character; path for saving intermediate MCMC output (default `"MCMC_save.rds"`).

`save.file2` Character; secondary save path used inside the RJMCMC helper (default `"MCMC_save.rds"`).

Details

Run reversible-jump MCMC for a single circadian dataset

Value

A named list of G x K matrices (K = floor(iteration / thin)):

rho Posterior samples of the rhythmicity indicator in {0, 1}.

phi Posterior samples of acrophase in hours (Zeitgeber scale).

A Posterior samples of amplitude.

M Posterior samples of MESOR.

sigma Posterior samples of residual variance.

log.r1, log.r1.SS, log.r3_A, log.r3_phi RJMCMC log-ratio components (for diagnostics).

if.accept.rj Binary matrix indicating accepted RJMCMC moves.

All matrices have `rownames` equal to `Data.list$gname`. After calling `match_symbols()`, the attributes `attr(rho, "symbols")` and `attr(rho, "RHYindex")` are set.

Examples

```
## Not run:
sim <- CBt_sim_data()
dat <- list(data  = as.data.frame(sim$data[[1]]$dat),
            time  = sim$data[[1]]$x$time,
            gname = paste0("G", seq_len(nrow(sim$data[[1]]$dat))))
init <- CB_init_single(dat)
mcmc_out <- CB_MCMC_single_rj_slice(dat, init,
              iteration = 200, thin = 10, n.burn = 100)

## End(Not run)
```

`CB_getAllEst` *Get all posterior estimates from BayRC MCMC output*

Description

Computes posterior estimates and highest-density credible intervals for amplitude (A), acrophase (phi), MESOR (M), residual variance (sigma), and optionally peak time (t_p) from the MCMC output list. Circular statistics (`circular_median` and `circular_HDI`) are used for phi/t_p; linear posterior means and HDI are used for the rest.

Usage

```
CB_getAllEst(
  res,
  burn = 100,
  CI = TRUE,
  credMass = 0.95,
  P = 24,
  rhythmic = FALSE
)
```

Arguments

`res`	Named list; MCMC output from `CB_MCMC_single_rj_slice` (or `CBt_MCMC_single`). Must contain matrices `rho`, `A`, `phi`, `M`, `sigma` with `attr(rho, "symbols")` and `attr(rho, "RHYindex")` set by `match_symbols`.
`burn`	Integer; number of initial MCMC iterations to discard as additional burn-in when computing estimates (default 100).
`CI`	Logical; if `TRUE` (default), return lower and upper credible-interval bounds in addition to the point estimate.
`credMass`	Numeric in (0, 1); nominal coverage of the HDI (default 0.95).
`P`	Numeric; period in hours used for circular statistics (default 24).
`rhythmic`	Logical; if `TRUE`, return only rows for genes classified as rhythmic (RHYindex == 1; default `FALSE`).

Details

Extract posterior estimates and credible intervals for all parameters

Value

A list of data.frames, one per parameter, each with columns `<param>.Est`, `<param>.Lower`, `<param>.Upper` and `RHYindex`. Row names are gene symbols when `attr(rho, "symbols")` is set.

Examples

```
## Not run:
sim  <- CBt_sim_data()
dat  <- list(data  = as.data.frame(sim$data[[1]]$dat),
             time  = sim$data[[1]]$x$time,
             gname = paste0("G", seq_len(nrow(sim$data[[1]]$dat))))
init <- CB_init_single(dat)
res  <- CB_MCMC_single_rj_slice(dat, init,
          iteration = 300, thin = 10, n.burn = 100)
est  <- CB_getAllEst(res, burn = 10)

## End(Not run)
```

`CBt_sim_data` *Simulate CBt circadian dataset*

Description

Generates synthetic multi-group circadian expression data following the BayRC cosinor model, optionally with Zeitgeber-time measurement error (t_p) on a specified proportion of samples. Each gene's true phase is drawn uniformly from \[0, P), and expression is generated as Y = M + A * cos(omega * (t + t_p) - phi) + epsilon. Supports multiple rhythmicity/phase-shift scenarios defined by `TOJR.grid`.

Usage

```
CBt_sim_data(
  P = 24,
  TOJR.grid = NULL,
  dParam = NULL,
  G = NULL,
  n = 30,
  fixed_tp = TRUE,
  sigma.t = 2,
  n.sigma.t = 16,
  p.sigma.t = NULL,
  Params = list(A = 0.5, phi = 0, M = 5, sigma = 1),
  TimePoints = NULL,
  PhaseClust = NULL
)
```

Arguments

`P`	Numeric; circadian period in hours (default 24).
`TOJR.grid`	Data.frame or matrix; G_scenario x n.group binary rhythmicity indicator grid defining which groups are rhythmic for each scenario.
`dParam`	Named list; group-level parameter offsets (`dA`, `dPhase`, `dM`) relative to the reference group, one element per non-reference group.
`G`	Integer vector of length n_scenario; number of genes per scenario row.
`n`	Integer; number of samples (default 30).

fixed_tp	Logical; if TRUE (default), time errors are fixed at +/- sigma.t; if FALSE, they are drawn from N(0, sigma.t).
sigma.t	Numeric; magnitude of the Zeitgeber-time measurement error (default 2).
n.sigma.t	Integer or NULL; number of samples with non-zero t_p. Exactly one of n.sigma.t and p.sigma.t must be non-NULL.
p.sigma.t	Numeric or NULL; proportion of samples with non-zero t_p.
Params	Named list; base cosinor parameters: A (amplitude), phi (phase), M (MESOR), sigma (residual SD).
TimePoints	Numeric vector or NULL; fixed observation times; if NULL, times are drawn uniformly from \[0, P).
PhaseClust	Currently reserved (default NULL).

Details

Simulate circadian expression data with Zeitgeber-time measurement error

Value

A named list with elements:

data A list of length n.group; each element contains x (sample-level covariates including time, time.p), beta (gene-level true parameters), and dat (G x N expression matrix).

TOJR Data.frame; expanded rhythmicity grid concatenated with parameter offsets for all genes.

Examples

```
## Not run:
sim <- CBt_sim_data(n.sigma.t = 8)

## End(Not run)
```

---

circular_HDI — *Compute the shortest circular highest-density interval*

Description

Finds the shortest interval on a circle of circumference P that contains at least credMass of the posterior samples. Uses a sliding-window algorithm on a doubled sorted array to handle wrap-around.

Usage

```
circular_HDI(samples, credMass = 0.95, P = 24, a = 0)
```

Arguments

samples	Numeric vector; phase samples in hours.
credMass	Numeric; desired coverage probability (default 0.95).
P	Numeric; period / circle circumference in hours (default 24).
a	Numeric; left endpoint of the normalisation interval (default 0).

Value

List with elements `lower` and `upper` (both in \[a, a + P)) giving the bounds of the shortest HDI.

`circular_median` *Compute the circular median of phase samples*

Description

Converts samples from hours to radians, computes the circular median via `circular::median.circular`, and converts back to hours.

Usage

```
circular_median(samples, P = 24)
```

Arguments

`samples`	Numeric vector; phase samples in hours.
`P`	Numeric; period in hours (default 24).

Value

Numeric scalar; circular median in hours in \[0, P).

`match_symbols` *Map Ensembl IDs to gene symbols and set rhythmicity attributes*

Description

Takes the raw list output from `CB_MCMC_single_rj_slice` and (optionally) converts Ensembl row names to HGNC gene symbols via biomaRt. If row names are already symbols they are used directly. Computes a Bayes Factor for each gene using `summarize_bay` and sets `attr(rho, "symbols")` and `attr(rho, "RHYindex")` so that downstream functions can access gene identifiers and rhythmicity classifications. Duplicate symbols are resolved by keeping the row with the highest Bayes Factor.

Usage

```
match_symbols(input_df, BF, p_rhythmic = 0.5, ensemble = NULL)
```

Arguments

`input_df`	Named list; raw MCMC output (e.g. from `CB_MCMC_single_rj_slice`). Must contain a `rho` matrix with gene identifiers as row names.
`BF`	Numeric; Bayes Factor threshold above which a gene is called rhythmic.
`p_rhythmic`	Numeric in (0, 1); prior probability of rhythmicity used to compute the Bayes Factor (default 0.5).
`ensemble`	A biomaRt Mart object; required when row names are Ensembl IDs (default `NULL`).

Details

Annotate MCMC output with gene symbols and rhythmicity index

Value

The input list filtered to unique symbols, with all matrix elements row-subsetted and `attr(*, "symbols")`, `attr(*, "RHYindex")`, and (if Ensembl IDs were present) `attr(*, "ensembl_gene_ids")` set on each matrix.

Examples

```
## Not run:
res  <- CB_MCMC_single_rj_slice(dat, init)
res2 <- match_symbols(res, BF = 3, p_rhythmic = 0.2)

## End(Not run)
```

`bfdr_from_posterior` *BFDR threshold from posterior rhythmicity probabilities*

Description

Implements the BFDR rule described in the BayRC paper (Eq. 25): sort genes by decreasing posterior probability p_g, compute the running average of (1 - p_g), and return the largest threshold tau_c such that BFDR(tau_c) = mean(1 - p_g | p_g >= tau_c) <= alpha. This controls the expected proportion of false rhythmic calls at level alpha.

Usage

```
bfdr_from_posterior(posterior_probs, alpha = 0.05)
```

Arguments

`posterior_probs`	Numeric vector; marginal posterior probability of rhythmicity for each gene (values in \[0, 1\]).
`alpha`	Numeric; desired BFDR level (default 0.05).

Details

Estimate the Bayesian False Discovery Rate threshold from posterior probabilities

Value

A list with elements:

threshold Numeric; the BFDR-optimal cutoff tau_c.

rhythmic_genes Logical vector; `TRUE` for called rhythmic.

n_rhythmic Integer; number of rhythmic calls.

bfdr_values Numeric vector; BFDR at each ranked position.

sorted_probs Sorted (descending) posterior probabilities.

detect_rhy *Detect rhythmic genes in two conditions using BFDR control*

Description

Applies `bfdr_from_posterior` separately to the marginal posterior rhythmicity probabilities of two conditions and returns the BFDR-optimal rhythmic gene sets for each, together with data.frames of annotated gene information.

Usage

```
detect_rhy(dat1, dat2, bfdr_alpha = 0.05)
```

Arguments

`dat1`	Named list; MCMC output for condition A with a `rho` matrix.
`dat2`	Named list; MCMC output for condition B with a `rho` matrix.
`bfdr_alpha`	Numeric; BFDR level (default 0.05).

Value

A list with elements:

rhythmic_A_logical, rhythmic_B_logical Logical vectors for conditions A and B.

rhythmic_A, rhythmic_B Data.frames of rhythmic genes with posterior probability columns.

threshold_A, threshold_B BFDR thresholds for each condition.

n_rhythmic_A, n_rhythmic_B, n_total, bfdr_alpha Counts and settings.

summarize_bay *Summarise posterior rhythmicity and Bayes Factor per gene*

Description

Computes the posterior mean rhythmicity (`RowAverage`), the number of zero samples (`Zeroes`), and the Bayes Factor for each gene. The Bayes Factor is computed as posterior odds / prior odds: BF = (rho_bar / (1 - rho_bar)) / (p_rhythmic / (1 - p_rhythmic)). Genes with BF > the threshold are called rhythmic (Rhythmicity = 1).

Usage

```
summarize_bay(input_df, BF, p_rhythmic = 0.2)
```

Arguments

`input_df`	G x K binary matrix of posterior rho samples.
`BF`	Numeric; Bayes Factor threshold for calling a gene rhythmic.
`p_rhythmic`	Numeric; prior probability of rhythmicity (default 0.5).

Value

Data.frame with columns `Zeroes`, `RowAverage`, `Rhythmicity` (0/1), and `BayesF`.

transition_classify

Bayesian transition classification (gain / loss / maintained)

Description

Computes joint transition posterior probabilities: p_gain = (1 - p_A) * p_B (gained rhythmicity), p_loss = p_A * (1 - p_B) (lost rhythmicity), p_cons = p_A * p_B (maintained rhythmicity), then applies `bfdr_from_posterior` to each to identify gain, loss, and conserved gene sets at the specified BFDR level.

Usage

```
transition_classify(pA, pB, bfdr_alpha = 0.05)
```

Arguments

`pA`	Numeric vector; marginal posterior rhythmicity probability for condition A (reference; length G).
`pB`	Numeric vector; marginal posterior rhythmicity probability for condition B (length G).
`bfdr_alpha`	Numeric; BFDR control level (default 0.05).

Details

Classify rhythmicity transitions using joint BFDR on transition posteriors

Value

A list with elements:

gain_loss_status Named character vector of length G with values `"Gain"`, `"Loss"`, `"Maintained"`, or `"Non-rhythmic"`.

gain_genes, loss_genes, cons_genes Integer indices of genes in each class.

tau_gain, tau_loss, tau_cons BFDR thresholds for each transition type.

n_gain, n_loss, n_cons Counts.

results Data.frame with per-gene classification details.

phase_infer	*Bayesian phase inference with BFDR-controlled shift/conservation flags*

Description

For genes classified as "Maintained" by `transition_classify` or `transition_classify_marginal`, computes the posterior distribution of the phase difference delta_phi = phi1 - phi2 (wrapped to [-P/2, P/2)), estimates the probability of a significant shift (|delta_phi| >= shift), and applies BFDR control to flag significantly shifted or conserved genes. Optionally computes the full circular HDI of delta_phi when `compute_hdi = TRUE`.

Usage

```
phase_infer(
  phi_matrix1,
  phi_matrix2,
  gain_loss_status,
  P = 24,
  credMass = 0.95,
  shift = 2,
  a = -P/2,
  bfdr_alpha = 0.05,
  compute_hdi = FALSE
)
```

Arguments

`phi_matrix1`	G x K matrix; posterior phase samples for condition 1.
`phi_matrix2`	G x K matrix; posterior phase samples for condition 2.
`gain_loss_status`	Named character vector of length G with values "`Gain`", "`Loss`", "`Maintained`", or "`Non-rhythmic`" (from `transition_classify`).
`P`	Numeric; period in hours (default 24).
`credMass`	Numeric; HDI coverage (default 0.95); only used when `compute_hdi = TRUE`.
`shift`	Numeric; phase-shift threshold in hours (default 4).
`a`	Numeric; left endpoint for circular normalisation (default `-P/2`).
`bfdr_alpha`	Numeric; BFDR control level (default 0.05).
`compute_hdi`	Logical; if `TRUE`, compute the circular HDI for each maintained gene (slower; default `FALSE`).

Details

Infer phase shifts and conservation among maintained rhythmic genes

Value

A list with per-gene vectors:

peak1, peak2 Circular median phases for each condition.

deltaPhi.Est, deltaPhi.Lower, deltaPhi.Upper Phase-difference median and HDI (filled only when `compute_hdi = TRUE`).

prob_shift, prob_conserved Posterior probabilities of shift and conservation.

flag_shift, flag_cons, flag_undetermined BFDR-significant flags.

BFDR_shift, BFDR_cons Running BFDR values.

`pathSelect` *FGSEA-based pathway selection for circadian transitions*

Description

Applies fast gene-set enrichment analysis (FGSEA) to a posterior gene-ranking metric derived from two BayRC MCMC outputs. Genes are ranked by their log-odds-ratio of gain vs loss, or by conservation, gain, loss, or union probability; pathways are tested for enrichment toward either end of this ranking. Pathway-level effect sizes and probabilistic gain/loss/conservation indices are added to the output.

Usage

```
pathSelect(
  mcmc.merge.list,
  pathway.list,
  dataset.names = NULL,
  ranking.method = c("log_odds_ratio", "union", "conserved", "gain", "loss"),
  score_type = c("std", "pos", "neg"),
  pathwaysize.lower.cut = 10,
  pathwaysize.upper.cut = 200,
  qvalue.cut = 0.05,
  epsilon = 0.001,
  n_top_genes = 5,
  nperm = 10000,
  nproc = 1,
  seed = 12345
)
```

Arguments

`mcmc.merge.list`
: Named list of exactly 2 MCMC output lists; the first is condition A (reference) and the second is condition B. Each must have `attr(rho, "symbols")` set.

`pathway.list` Named list of character vectors; pathway gene sets.

`dataset.names`
: Character vector of length 2 or `NULL`; labels for conditions A and B (default auto-detected from list names).

ranking.method
: Character; gene-ranking metric: one of "log_odds_ratio" (default), "union", "conserved", "gain", or "loss".

score_type
: Character; fgsea score type: "std" (two-sided), "pos" (gain only), or "neg" (loss only, inverts ranking).

pathwaysize.lower.cut
: Integer; minimum pathway size (default 10).

pathwaysize.upper.cut
: Integer; maximum pathway size (default 200).

qvalue.cut
: Numeric; adjusted p-value significance cutoff (default 0.05).

epsilon
: Numeric; small constant added to probabilities to avoid log(0) (default 0.001).

n_top_genes
: Integer; number of top genes to report per category per pathway (default 5).

nperm
: Integer; fgsea permutations (default 10000).

nproc
: Integer; fgsea parallel processes (default 1).

seed
: Integer; random seed (default 12345).

Details

Select circadian-enriched pathways using FGSEA

Value

A list with elements:

results Data.frame of fgsea results (one row per pathway) augmented with Gain_Index, Loss_Index, Conserved_Index, Gain_Loss_Ratio_Arithmetic, expected counts, and top gene columns.

gene_rankings Data.frame of per-gene ranking metrics and posterior probabilities.

parameters List of analysis settings.

Examples

```
## Not run:
res <- pathSelect(list(human = human_res, mouse = mouse_res),
                  pathway.list = msig_pathways,
                  ranking.method = "log_odds_ratio")
head(res$results)

## End(Not run)
```

---

plot_heatmap *Pathway-level expression heatmap*

Description

Produces a ComplexHeatmap visualisation for a given pathway, showing posterior rhythmicity probabilities and phase information for both conditions side by side. Genes can be coloured by gain/loss/ maintained status from transition_classify output. The plot can show all genes, only rhythmic genes, or both (two panels).

Usage

```
plot_heatmap(
  data1,
  data2,
  pathway_genes,
  pathway_name,
  phase_results,
  transition_results = NULL,
  group_names = c("Group1", "Group2"),
  save_path = NULL,
  n_bins = 24,
  versions = c("full", "rhythmic_only", "both")
)
```

Arguments

`data1`	Named list; MCMC output for condition 1 with `rho` and `phi` matrices.
`data2`	Named list; MCMC output for condition 2.
`pathway_genes`	Character vector; gene names in the pathway.
`pathway_name`	Character; pathway label used in the plot title.
`phase_results`	Output from `phase_infer` or a similar list with per-gene phase metrics.
`transition_results`	Output from `transition_classify` or `NULL` (default `NULL`); used to colour genes by gain/loss/maintained status.
`group_names`	Character vector of length 2; condition labels (default `c("Group1", "Group2")`).
`save_path`	Character or `NULL`; file path for saving the plot PNG; if `NULL` the plot is drawn but not saved.
`n_bins`	Integer; number of bins for the phase colour wheel (default 24).
`versions`	Character; which version to produce: `"full"`, `"rhythmic_only"`, or `"both"` (default `"full"`).

Details

Integrated pathway heatmap with rhythmicity and phase information

Value

Called for side effects; invisibly returns the heatmap object.

`multi_conservation_pathway` *Multi-condition pathway circadian conservation analysis*

Description

Runs all pairwise pathway conservation comparisons among a list of MCMC outputs: computes observed pathway congruence scores, generates permutation null distributions, applies BH correction, and writes results to an Excel file.

Usage

```
multi_conservation_pathway(
  mcmc.merge.list,
  dataset.names,
  select.pathway.list,
  delta = 3,
  units = "hours",
  B = 1000,
  ncores = NULL,
  parallel = "auto",
  rounds = 1,
  save_intermediate = FALSE,
  intermediate_dir = "intermediate_results",
  output.dir = "Conservation_Pathway"
)
```

Arguments

`mcmc.merge.list`	Named list of MCMC output lists, one per condition.
`dataset.names`	Character vector; condition labels.
`select.pathway.list`	Named list of character vectors; pathway gene sets.
`delta`	Numeric; phase threshold (default 3).
`units`	Character; `"hours"` (default).
`B`	Integer; permutations per pair (default 1000).
`ncores`	Integer or `NULL`; parallel cores.
`parallel`	Logical or `"auto"`.
`rounds`	Integer; rounds per pair.
`save_intermediate`	Logical; save intermediate results.
`intermediate_dir`	Character; directory for intermediate saves.
`output.dir`	Character; directory for Excel output (default `"Conservation_Pathway"`).

Details

Pairwise pathway conservation analysis across multiple conditions

Value

Named list of pairwise pathway result data.frames. Also writes an Excel file to `output.dir`.

`multi_conservation_pathway_bootstrap`
Pathway-level bootstrap concordance (compatibility wrapper)

Description

Thin wrapper around `multi_conservation` retained for backward compatibility with analysis scripts that call `multi_conservation_pathway_bootstrap`. All computation is delegated to `multi_conservation`; the `delta` and `units` arguments are accepted but unused (the c-score is threshold-free by design; see paper Eq. 3).

Note: output Excel sheets are named `"Results"` and `"Column_Definitions"`, not `"congruence_index"` as in the legacy Thien/ version. Update downstream read.xlsx calls accordingly.

Usage

```
multi_conservation_pathway_bootstrap(
  mcmc.merge.list,
  dataset.names,
  select.pathway.list,
  delta = 3,
  units = "hours",
  n_boot = 500,
  n_perm = 1000,
  output.dir = "Conservation_Pathway_Bootstrap",
  ...
)
```

Arguments

`mcmc.merge.list`	Named list of MCMC outputs.
`dataset.names`	Character vector of dataset labels.
`select.pathway.list`	Named list of pathway gene sets.
`delta`	Numeric; ignored (reserved for future phase-threshold concordance variant).
`units`	Character; ignored.
`n_boot`	Integer; bootstrap replicates (default 500).
`n_perm`	Integer; permutation replicates (default 1000).
`output.dir`	Character; directory for Excel output.
`...`	Additional arguments passed to `multi_conservation`.

Value

Same value as `multi_conservation`.

multi_conservation *Multi-dataset pairwise pathway concordance analysis*

Description

Runs all pairwise pathway bootstrap concordance analyses across a list of MCMC outputs and compiles results into Excel output. Wraps `bootstrap_conservation_pathway_within`.

Usage

```
multi_conservation(
  mcmc.merge.list,
  dataset.names,
  select.pathway.list,
  n_perm = 1000,
  n_boot = 500,
  alpha = 0.05,
  min.p = 5,
  qvalue_threshold = NULL,
  output.dir = "Conservation_Pathway_Bootstrap",
  save_output = TRUE,
  use_cpp = FALSE,
  compute_pvalue = TRUE,
  compute_ci = TRUE
)
```

Arguments

`mcmc.merge.list`
: Named list of MCMC output lists.

`dataset.names`
: Character vector; condition labels.

`select.pathway.list`
: Named list of pathway gene sets.

`n_perm`
: Integer; permutations per pathway (default 1000).

`n_boot`
: Integer; bootstrap replicates per pathway (default 500).

`alpha`
: Numeric; significance level (default 0.05).

`min.p`
: Integer; minimum number of pathway genes required to test a pathway (default 5).

`qvalue_threshold`
: Numeric or `NULL`; q-value significance cutoff.

`output.dir`
: Character; directory for Excel output.

Value

Named list of pairwise result data.frames. Also writes an Excel file to `output.dir`.

pairwise_concordance *Compute pairwise concordance matrix across multiple tissues*

Description

For each pair of tissues in `human_data`, computes the Jaccard concordance index between their posterior rhythmicity classifications. Optionally restricts the comparison to the top `n_gene` genes ranked by mean posterior rhythmicity across all tissues.

Usage

```
pairwise_concordance(human_data, n_gene = NULL)
```

Arguments

`human_data`	Named list of MCMC output lists; each element must have a `rho` matrix with matching gene row names.
`n_gene`	Integer or `NULL`; if specified, restrict to the top `n_gene` genes by mean rhythmicity across tissues.

Value

Square symmetric numeric matrix of Jaccard concordance values, with row and column names equal to `names(human_data)`.

match_homologs *Match homologs across species MCMC outputs*

Description

Uses biomaRt to retrieve one-to-one orthologs between each non-reference species and the reference species (default human), then intersects the mapped gene sets across all datasets. Each input list is row-filtered and reordered so that all datasets share the same genes in the same order, enabling direct cross-species comparisons.

Usage

```
match_homologs(input_dfs, species_from, ref = "human")
```

Arguments

`input_dfs`	A list of MCMC output lists (one per dataset), each with matrices whose row names are Ensembl gene IDs and whose `attr(rho, "ensembl_gene_ids")` attribute is set.
`species_from`	Character vector of the same length as `input_dfs`; species label for each dataset (currently supports `"human"` and `"mouse"`).
`ref`	Character; reference species for the common gene space (default `"human"`).

Details

Align multiple MCMC outputs to a common set of homologous genes

Value

A list of the same length as `input_dfs`, each element row-filtered to the intersection of homologous reference-space gene IDs and reordered consistently.

Examples

```
## Not run:
aligned <- match_homologs(list(human_res, mouse_res),
                          species_from = c("human", "mouse"))

## End(Not run)
```

`merge_mcmc` *Merge MCMC output lists by orthologs*

Description

Intersects gene sets across a list of MCMC outputs from different species or conditions. When `ortholog.db` is supplied and multiple species are present, the ortholog database maps each species' genes to a common reference gene space (defined by the `reference` dataset). When datasets share a single species (or no ortholog database is provided), the function simply takes the intersection of gene names. If `uniqG` = `TRUE`, duplicate ortholog matches are resolved by keeping the entry with the highest mean absolute posterior signal.

Usage

```
merge_mcmc(mcmc.list, species, ortholog.db = NULL, reference = 1, uniqG = T)
```

Arguments

`mcmc.list`	A list of MCMC output matrices or lists, one per condition or study. Row names must be gene identifiers.
`species`	Character vector of the same length as `mcmc.list`; species label for each dataset.
`ortholog.db`	Data.frame or matrix with one column per species (column names matching `species`); each row maps orthologous genes across species. If `NULL`, datasets are merged by gene name intersection only (default `NULL`).
`reference`	Integer; index of the reference dataset whose gene names define the merged row names (default 1).
`uniqG`	Logical; if `TRUE` (default), keep only the highest-signal match when multiple orthologs map to the same reference gene.

Details

Merge multiple MCMC outputs by matching orthologous genes

Value

A list of the same length as `mcmc.list`, each element row-subsetted and reordered to the common gene set, with row names from the reference dataset.

Examples

```
## Not run:
merged <- merge_mcmc(list(human_res, mouse_res),
                     species      = c("human", "mouse"),
                     ortholog.db  = hm_orth,
                     reference    = 1)

## End(Not run)
```

Index